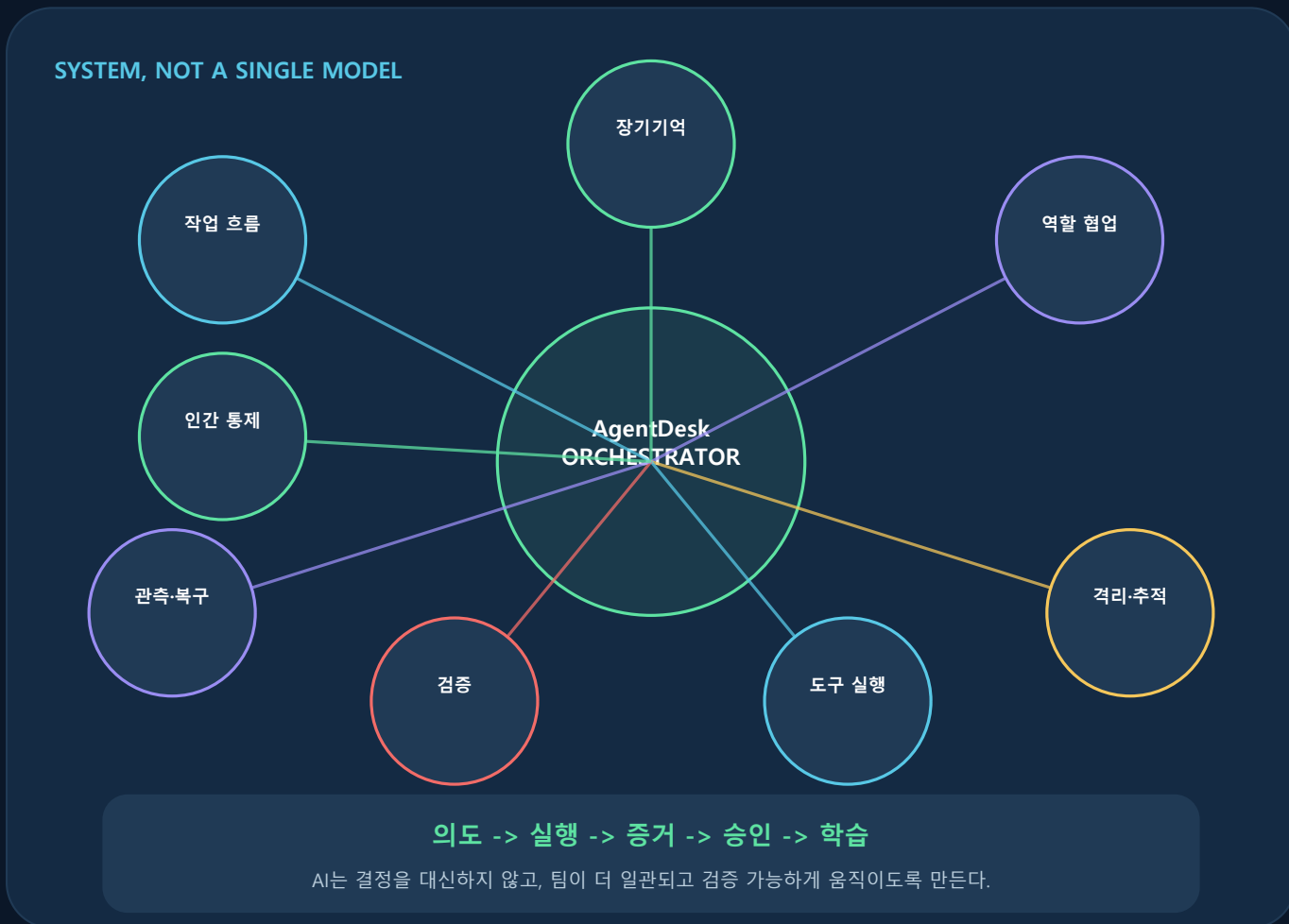


DoodleUp

AI 개발 시스템 기술서

작업 흐름과 범용 기능을 중심으로 설명하는 AI-native 개발 운영 체계



한 테마는 하나의 질문만 책임진다

기능을 겹쳐 설명하지 않고, 시스템의 각 책임을 독립된 관점으로 나눈다.

SYSTEM THESIS

사람이 방향을 정하고, AI 시스템이 실행의 연속성·속도·증거를 만든다.

01

오케스트레이션

무엇이 언제 움직이는가

카드·의존성·상태

02

장기 기억

무엇이 세션 뒤에도 남는가

결정·오류·절차

03

역할 협업

누가 어떤 기준으로
판단하는가

소유권·handoff·review

04

격리·추적

어디서 안전하게 변경하는가

worktree·diff·commit

05

도구 실행

의도가 어떻게 산출물이
되는가

inspect·edit·execute

06

검증

완료를 어떻게 증명하는가

gate·evidence·QA

07

관측·복구

장시간 작업을 어떻게
통제하는가

state·log·resume·budget

08

인간 통제

최종 권한과 책임은 누구에게
있는가

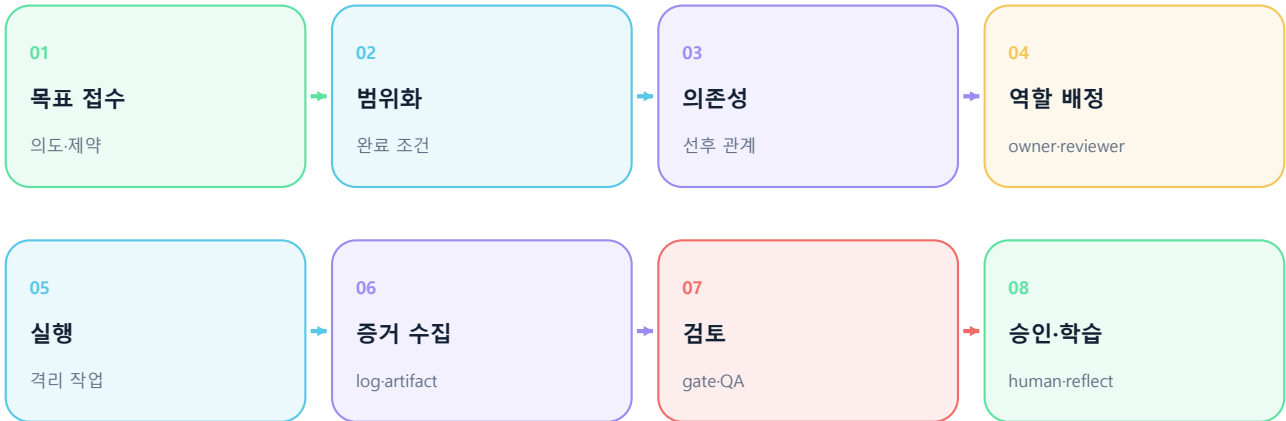
approval·safety·rights

설명 규칙 | 각 장은 작동 방식 -> 핵심 기능 -> 고유한 장점 순서로만 구성

무엇이 언제 움직이는가

목표를 카드로 바꾸고, 의존성과 상태를 관리하며, 적절한 역할과 실행 순서를 연결한다.

END-TO-END WORKFLOW



CARD STATE



기능	작동 방식	고유한 장점
Kanban 상태	실제 실행 상태와 보드를 맞춘다	진행과 대기를 혼동하지 않는다
의존성 graph	선행 작업이 닫힌 뒤 후속 작업을 연다	재작업과 충돌을 줄인다
수용 기준	시작 전에 완료 조건과 금지 범위를 적는다	scope drift를 억제한다
자원 인식 배정	역할뿐 아니라 shared resource 비용을 본다	안전한 병렬성과 처리량을 얻는다

예측 가능성

다음 상태와 종료 조건이 보인다.

조정 비용 감소

누가 무엇을 기다리는지 자동으로 드러난다.

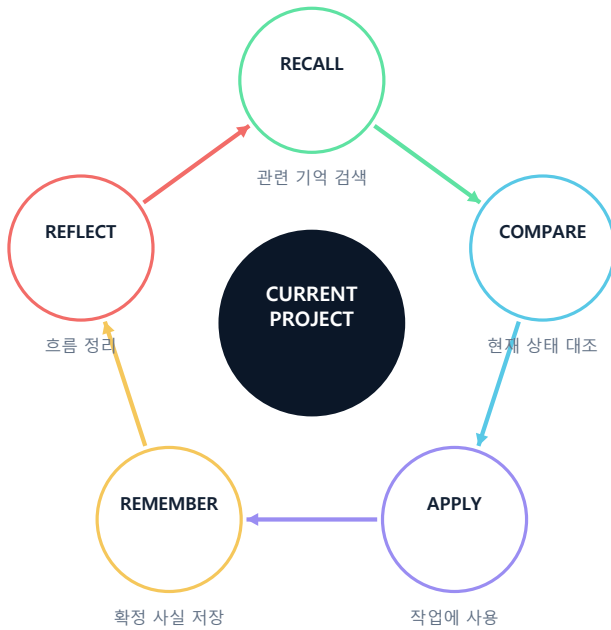
안전한 병렬화

의존성과 자원을 함께 보고 실행한다.

무엇이 세션 뒤에도 남는가

AnchorMind/Memento가 결정·오류·절차·선호를 검색 가능한 프로젝트 기억으로 유지한다.

MEMORY LIFECYCLE



기억 단위

fact

확인된 사실

decision

선택과 근거

error

재현·원인·해결

preference

사용자 선호

procedure

재실행 절차

relation

항목 간 연결

episode

목표·사건·결과

범위

저장 대상

고유한 장점

프로젝트 scope

공용 결정·절차

팀 전체가 같은 기준을 사용

역할 전용 scope

전문 role의 작업 문맥

불필요한 기억 혼합을 줄임

세션 scope

임시 가설·진행 정보

장기 기억 오염을 방지

가드레일

기억은 현재 코드보다 우선하지 않는다 · 비밀값과 개인정보는 저장하지 않는다 · 추측과 임시 log는 장기 기억에서 제외한다.

문맥 연속성

긴 프로젝트를 다시 설명하는 비용이 줄어든다.

실패 재사용

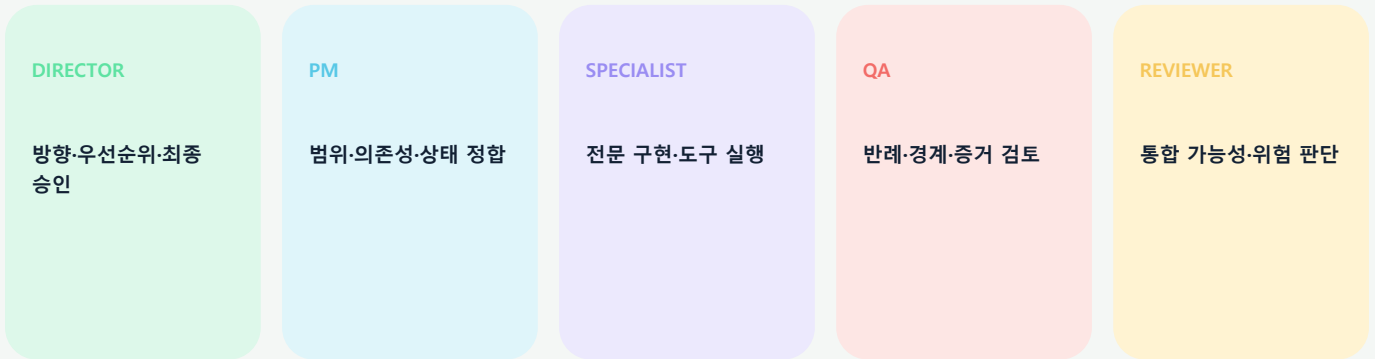
해결한 오류와 절차를 다음 작업에서 회수한다.

prompt 절약

필요한 기억만 검색해 context를 작게 유지한다.

누가 어떤 기준으로 판단하는가

역할은 model 수를 늘리는 장치가 아니라 판단 기준·소유권·review 책임을 분리하는 장치다.



HANDOFF CONTRACT



handoff에는 설명뿐 아니라 다음 역할이 판정할 수 있는 산출물과 증거가 포함된다.

기능	작동 방식	고유한 장점
단일 owner	카드마다 구현 책임자를 한 명 둔다	책임 공백과 중복 작업을 막음
독립 reviewer	구현자와 다른 기준으로 결과를 본다	확증 편향과 자기 승인 감소
명시적 handoff	입력·출력·완료 조건을 함께 전달한다	역할 사이 정보 손실 감소
human escalation	범위·권리·감각 판단은 사람에게 올린다	자동화의 권한 과잉 방지

전문화

각 역할이 자신의 판단 기준에 집중한다.

맹점 감소

구현과 검토를 다른 관점으로 분리한다.

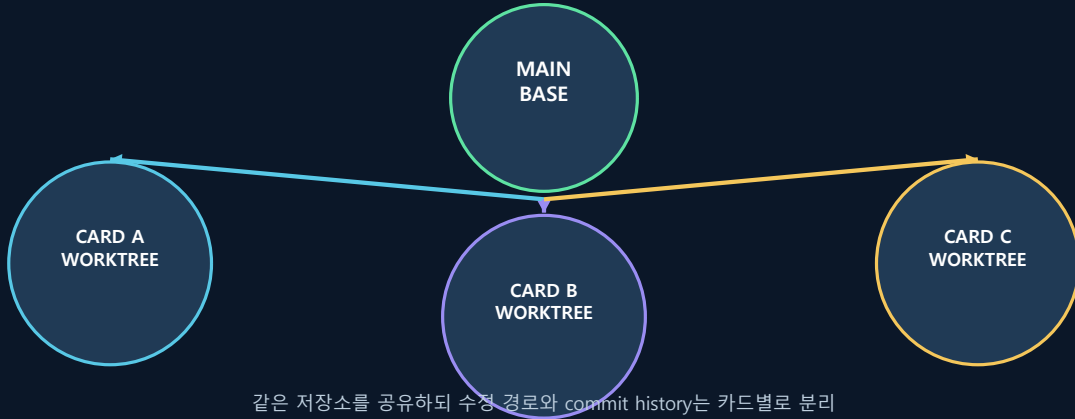
소유권 명확화

누가 결정하고 누가 승인하는지 남는다.

어디서 안전하게 변경하는가

카드별 branch와 worktree가 변경을 격리하고, diff-test-commit이 변경의 출처를 보존한다.

PARALLEL WORK, SEPARATE FILESYSTEMS



PROVENANCE CHAIN



주의 | worktree는 파일을 격리하지만 CPU-memory-engine cache 같은 공유 자원까지 격리하지는 않는다.

기능	작동 방식	고유한 장점
변경 격리	다른 카드의 미완성 변경과 섞지 않는다	오염 없는 review와 선택적 통합
원자 commit	하나의 기능 단위와 증거를 함께 기록	rollback과 원인 추적이 쉬움
명시적 staging	변경 경로를 지정해 필요한 파일만 포함	사용자 작업과 타 agent 변경을 보호
통합 gate	diff와 검증 결과를 보고 main에 반영	merge 자체가 승인 기록이 됨

안전한 동시 작업

서로의 미완성 변경을 직접 덮지 않는다.

복구 가능성

작은 commit 단위로 되돌리고 비교한다.

감사 가능성

카드·변경·검증·통합이 한 이력으로 연결된다.

의도가 어떻게 실제 산출물이 되는가

에이전트는 답변만 생성하지 않고, 저장소와 제작 도구를 inspect-edit-execute-capture 순서로 사용한다.

TOOL INVOCATION CYCLE



도구 결과는 다음 판단의 입력이 되고, 실패하면 log를 근거로 범위를 줄여 다시 실행한다.

REPOSITORY

search · diff · patch · Git

RUNTIME

compile · test · build · editor

CONTENT

asset · scene · document

CONNECTORS

memory · issue · communication

기능	작동 방식	고유한 장점
상태 기반 실행	먼저 현재 파일·process·tool 상태를 읽는다	추측성 수정과 잘못된 대상 작업 감소
작은 patch	변경 범위를 최소화하고 diff로 확인한다	review와 rollback 비용 감소
machine-readable 출력	가능하면 JSON·XML·log·hash로 결과를 남긴다	다른 agent와 사람이 재검증 가능
권한 경계	배포·외부 전송·파괴적 변경은 승인 후 실행	자동화가 scope 밖으로 확장되는 것을 방지

답변에서 산출물로

자연어가 실제 파일·build·report로 이어진다.

반복 가능성

같은 절차를 다시 실행하고 비교할 수 있다.

handoff 감소

사람이 도구 사이에서 결과를 옮기는 일이 줄어든다.

완료를 어떻게 증명하는가

자연어 완료 선언을 여러 층의 자동 gate와 독립 review가 확인할 수 있는 evidence bundle로 바꾼다.

LAYERED GATES

HUMAN

감각·우선순위·출시

INDEPENDENT REVIEW

수용 기준·반례

INTEGRATION

전체 흐름·상호작용

TARGETED TEST

상태·경계값

COMPILE / STATIC 문법·형식·기본 무결성

EVIDENCE BUNDLE

- INPUT 요청·수용 기준
- CHANGE diff·artifact
- RUN command·환경
- RESULT log·raw output
- IDENTITY commit·hash
- REVIEW 판정·한계

판정 주체	담당 범위	고유한 장점	경계
자동 gate	compile-test-validator-build	빠르고 반복 가능	재미·감각은 판단 못함
독립 review	수용 기준·raw evidence·edge case	자기 승인 위험 감소	review 품질에 의존
인간 승인	감각·범위·권리·출시 판단	목적과 책임 유지	시간과 주의가 필요

핵심 | test PASS는 기술적 위험을 줄였다는 뜻이지, 결과가 재미있거나 옳다는 뜻은 아니다.

거짓 완료 감소

완료 선언에 재현 가능한 증거가 필요하다.

회귀 조기 발견

작은 gate부터 실패 지점을 좁힌다.

신뢰 형성

사람과 agent가 같은 evidence를 보고 판단한다.

장시간 작업을 어떻게 통제하는가

실행 상태-queue-log-heartbeat-context budget을 분리해 running, waiting, stuck를 구별하고 재개한다.

CONTROL SURFACE

STATE

active · idle · blocked

QUEUE

waiting · dependency

PROGRESS

tool call · artifact

LOG

stdout · error · summary

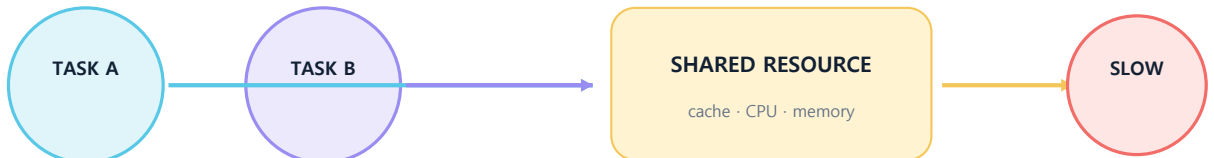
HEARTBEAT

alive · recover

BUDGET

context · image · cost

RESOURCE CONTENTION PATTERN



파일 격리와 실행 자원 격리는 다르므로 concurrency보다 resource affinity를 먼저 본다.

기능	작동 방식	고유한 장점
진행 상태	card와 session 상태를 별도 표시	작업 중-대기-정지를 구별
log-first	큰 출력은 파일로 남기고 요약만 context에 넣음	문맥 팽창과 재확인 비용 감소
checkpoint-resume	card-worktree-commit-memory에서 재개	중단 뒤 처음부터 반복하지 않음
watchdog	heartbeat와 실패 상태로 재시작-escalation	장시간 무응답을 방지하지 않음
budget policy	image-context-tool time에 상한과 우선순위	비용과 latency를 예측 가능하게 함

상태 가시성

실제로 느린지, 기다리는지, 멈췄는지 안다.

복구력

중단된 지점의 증거와 문맥에서 이어간다.

자원 효율

공유 자원과 context를 기준으로 동시성을 조절한다.

최종 권한과 책임은 누구에게 있는가

AI가 실행 범위를 넓혀도 목표·우선순위·감각·외부 공개·권리 판단은 인간의 승인 아래 둔다.

AUTHORITY MAP

목표·우선순위

HUMAN

작업 분해·실행

AI SYSTEM

기계 검증

AUTOMATION

감각·품질 승인

HUMAN

외부 공개·배포

HUMAN

권리·license 판단

HUMAN

통제 항목	작동 방식	고유한 장점
비밀정보	token-password-개인정보를 memory-repo-prompt에서 제외	노출 위험 감소
범위 통제	파괴적 변경·배포·외부 전송은 명시적 승인 후 수행	자동화의 권한 과잉 방지
AI 공개	주요 AI 도구명과 활용 방식을 제출물에 고지	투명성과 규정 준수
asset ledger	원본·도구·모델·날짜·license·수정자·경로 기록	권리 provenance 확보
제출 경계	기존 IP와 행사 기간 신규 결과물을 tag-README로 구분	권리와 성과 범위 명확화

NAN 2026 TERMS

주요 AI 도구·활용 방식 고지 · 생성형 AI·open source·외부 API license 준수 · 제출 권리 확인

<https://nan2026.nhn.com/terms>

FINAL CLAIM

DoodleUp의 AI 개발 시스템은 사람을 대체하는 자동 제작기가 아니다. 작업을 구조화하고, 문맥을 보존하고, 전문 역할을 연결하고, 실행 결과를 증거로 바꾸면서도 최종 권한은 사람에게 남기는 개발 운영 체계다.